

Neural Networks & Deep Learning

Team SC_2

Vit:

- **Architecture Description:**

1. Input: Images resized to $224 \times 224 \times 3$
2. Conv2D (32 filters, 3×3 , ReLU) $\rightarrow$ BatchNorm $\rightarrow$ MaxPooling (2×2)
3. Conv2D (64 filters, 3×3 , ReLU) $\rightarrow$ BatchNorm $\rightarrow$ MaxPooling (2×2)
4. Conv2D (128 filters, 3×3 , ReLU) $\rightarrow$ BatchNorm $\rightarrow$ MaxPooling (2×2)
5. Flatten Layer
6. Dense (256 units, ReLU) $\rightarrow$ Dropout (0.5)
7. Dense (128 units, ReLU) $\rightarrow$ Dropout (0.5)
8. Dense (N classes, Softmax)

- **Trail:**

1. Lower learning rate allowed finer weight updates.
2. Early stopping prevented overfitting.
3. Dropout and BatchNorm stabilized training.
4. Adding too many layers increased overfitting without data.
5. High learning rate caused validation accuracy to fluctuate.
6. Kaggle Score: ~36.5%
7. val acc 0.4810

Efficient-Net0:

- **Architecture Description:**

1. Conv2D (3 → 32, kernel=3×3, stride=2) → BatchNorm → Swish activation
2. Convolutional Block Attention Module (CBAM)
3. Channel Attention: SE-like squeeze-excitation with Swish activation
4. Spatial Attention: Conv layer on concatenated avg/max maps
5. Applied after depthwise convolution in each MBConv block
6. Head Layers:
 - a. Conv2D (320 → 1280) → BatchNorm → Swish
 - b. Global Average Pooling → Dropout (0.5) → Linear (1280 → 17 classes)

- **Reason for Choosing This Architecture:**

1. Replicates EfficientNet-B0 structure for strong feature extraction.
2. Integrates CBAM for attention-based feature refinement.

- **Trails:**

Trial	Changes Made	Train Acc	Val Acc	Kaggle Score	Observations
1	Baseline model, OneCycleLR, class weights, label smoothing	9.19%	10.25%	—	Very low starting accuracy (random init)
2	Added strong augmentation (ColorJitter, RandomCrop)	13.92%	16.02%	—	Gradual improvement
3	Increased LR max to 2e-3, added weight decay 1e-3	20.61%	22.02%	—	Better convergence
4	Extended training to 50 epochs, early stopping patience=12	51.82%	34.32%	~0.34	Best val accuracy reached
5	Phase II fine-tuning (LR=1e-5, 10 epochs)	51.83%	34.32%	0.3432	No further improvement; early stopped

1. Loss Function: CrossEntropyLoss with class weights + label smoothing (0.1)
2. Optimizer: AdamW (weight decay=1e-3)
3. Epochs: 50 (Phase I) + 3 (Phase II fine-tuning)
4. Kaggle Score: ~24%
5. validation accuracy: 0.3432

Mobile-Net:

- **Architecture Description:**

1. Depthwise Separable Convolution Blocks: 3×3 depthwise convolution followed by 1×1 pointwise convolution
2. Squeeze-and-Excitation (SE) Attention: Global average pooling → Dense layers with ReLU/sigmoid → channel-wise scaling
3. Residual Connections: Skip connections with 1×1 convolution for dimension matching
4. Initial Convolution: Conv2D (3 → 16, kernel=3×3, stride=2) → BatchNorm → ReLU
5. Residual Block 1: Depthwise SepConv (16 → 32, stride=1) + SE Attention
6. Residual Block 2: Depthwise SepConv (32 → 64, stride=2) + SE Attention
7. Followed by identical block (stride=1)
8. Residual Block 3: Depthwise SepConv (64 → 128, stride=2) + SE Attention
9. Followed by identical block (stride=1)

10. Deep Stack: Depthwise SepConv ($128 \rightarrow 256$, stride=2) + SE Attention

11. $5\times$ identical residual blocks (stride=1)

12. Head: Global Average Pooling $\rightarrow$ Dropout (0.5) $\rightarrow$ Dense ($256 \rightarrow 17$ classes, softmax)

- **Reason for Choosing This Architecture:**

1. MobileNet-inspired efficiency: Depthwise separable convolutions reduce parameters/computations by 8-9 $\times$ compared to standard convolutions
2. Attention mechanism: SE blocks adaptively recalibrate channel-wise feature responses

- **Trail:**

1. Standard convolutional layers, Result: Severe overfitting $\rightarrow$ needed more efficient architecture
2. Adding L2 regularization ($1e-4$) for better generalization
3. Cosine decay LR schedule
4. Efficient training (mixed precision + cosine LR)
5. Kaggle Score: $\sim 22\%$
6. val_accuracy: 0.2917

Custom CNN:

- **Architecture Description:**

1. SimpleConvBlock: Conv2D $\rightarrow$ BatchNorm $\rightarrow$ SiLU activation
2. Sequential Feature Extractor: 8-layer deep CNN with progressive downsampling
3. Initial Conv Block: Conv2D ($3 \rightarrow 32$, kernel= 3×3 , stride=2)
4. Feature Extraction:
 - a. Conv2D ($32 \rightarrow 32$) $\rightarrow$ BatchNorm $\rightarrow$ SiLU
 - b. Conv2D ($32 \rightarrow 64$, stride=2) $\rightarrow$ BatchNorm $\rightarrow$ SiLU
 - c. Conv2D ($64 \rightarrow 64$) $\rightarrow$ BatchNorm $\rightarrow$ SiLU
 - d. Conv2D ($64 \rightarrow 128$, stride=2) $\rightarrow$ BatchNorm $\rightarrow$ SiLU
 - e. Conv2D ($128 \rightarrow 128$) $\rightarrow$ BatchNorm $\rightarrow$ SiLU
 - f. Conv2D ($128 \rightarrow 256$, stride=2) $\rightarrow$ BatchNorm $\rightarrow$ SiLU
 - g. Conv2D ($256 \rightarrow 256$) $\rightarrow$ BatchNorm $\rightarrow$ SiLU
5. Head:
 - a. Global Average Pooling $\rightarrow$ Flatten
 - b. Dropout (0.3) $\rightarrow$ Linear ($256 \rightarrow 256$) $\rightarrow$ SiLU
 - c. Dropout (0.3) $\rightarrow$ Linear ($256 \rightarrow 17$ classes)

- **Trail:**

1. 3-4 convolutional layers, Underfitting $\rightarrow$ needed more capacity
2. More blocks, Overfitting + slow training $\rightarrow$ scaled back to 8 layers

3. SiLU provided best convergence
4. Double dropout (0.3 each) after pooling and before final layer worked best
5. Kaggle Score: ~23%
6. Acc 0.3347